

Networks and Systems Management



Python Automation
Filipe Sousa
FEUP – GRS/M.EIC

Introduction to Linux Networking

Overview of networking in Linux

Importance of networking for
system administration and
development

Key concepts to be covered



User-Space vs. Kernel-Space

Kernel-Space

Handles low-level networking operations

Network stack, device drivers, and protocol handling

Uses syscalls for communication with user-space

User-Space

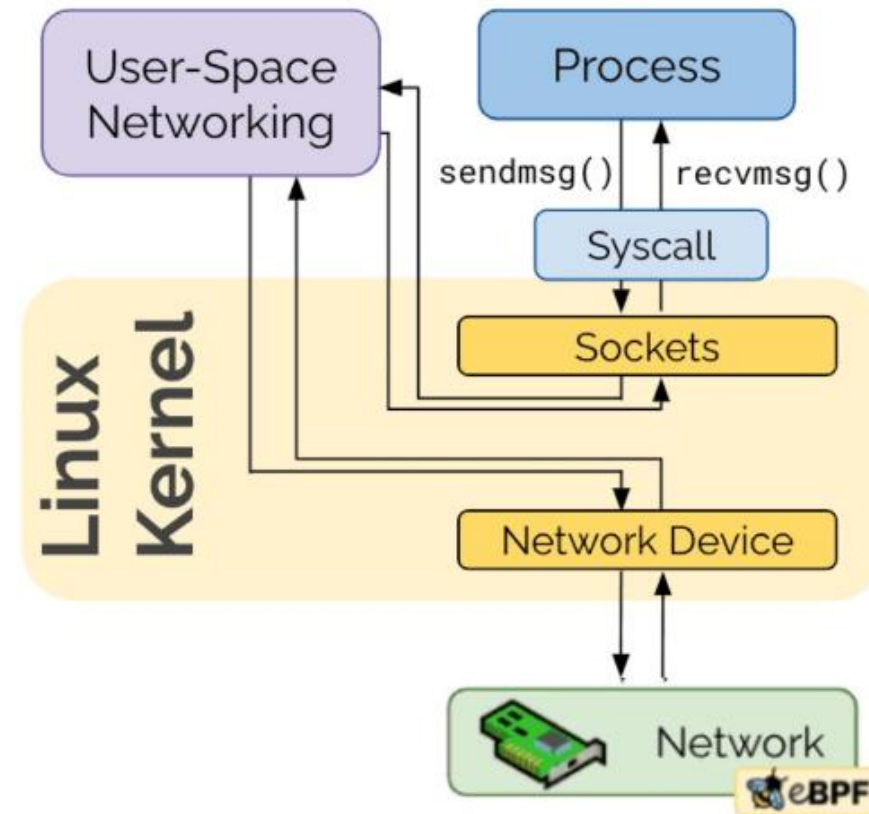
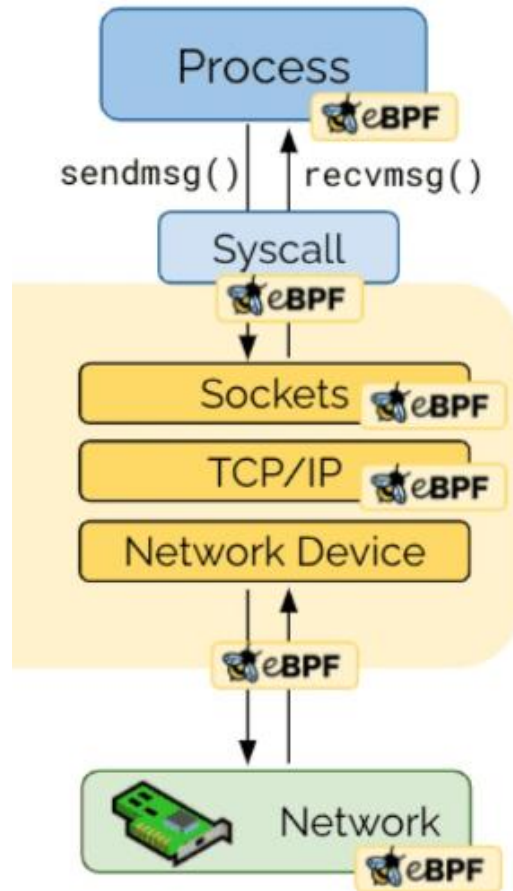
Network applications operate here (e.g., curl, ping, netcat)

Communicates with the kernel using system calls

Libraries like libpcap for packet capturing

User-Space vs. Kernel-Space

<https://cilium.io/blog/2020/11/10/ebpf-future-of-networking/>



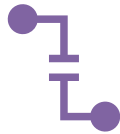
Routing in Linux



Routing Basics

Directing network packets
to their destination

Static vs. dynamic routing



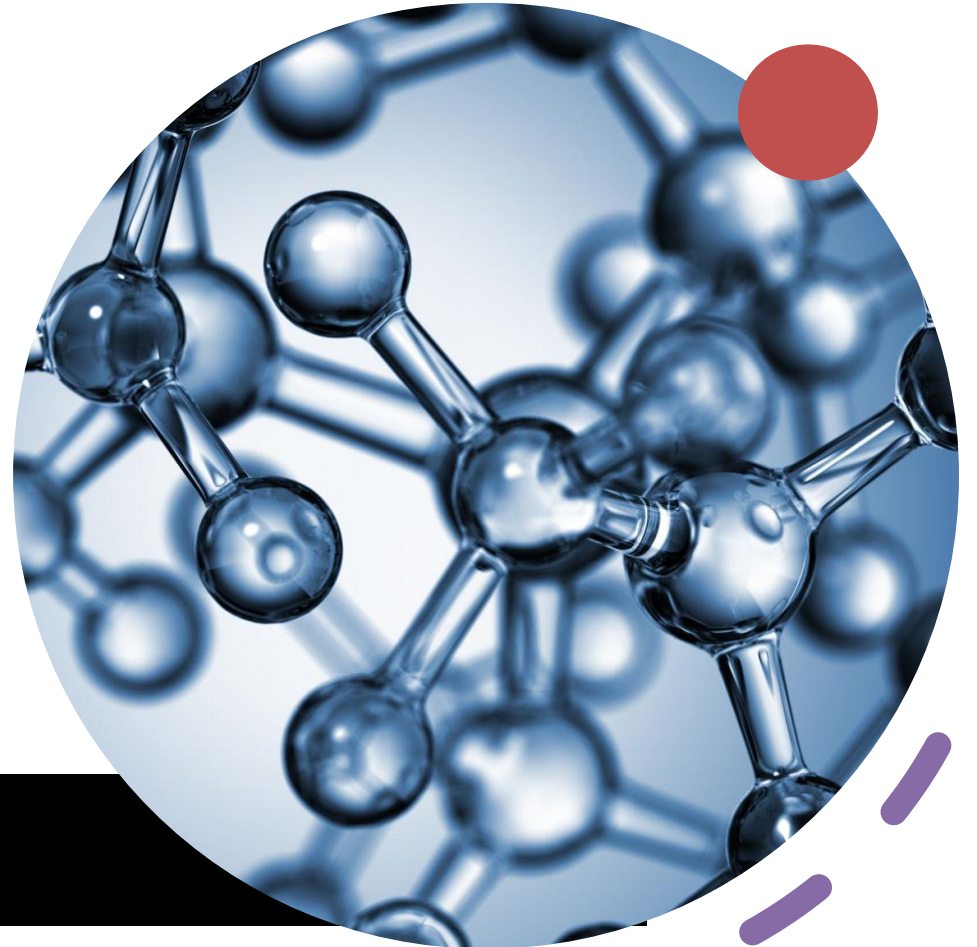
Linux Routing Tools

ip route for managing
routes

route command
(deprecated, but still in use)

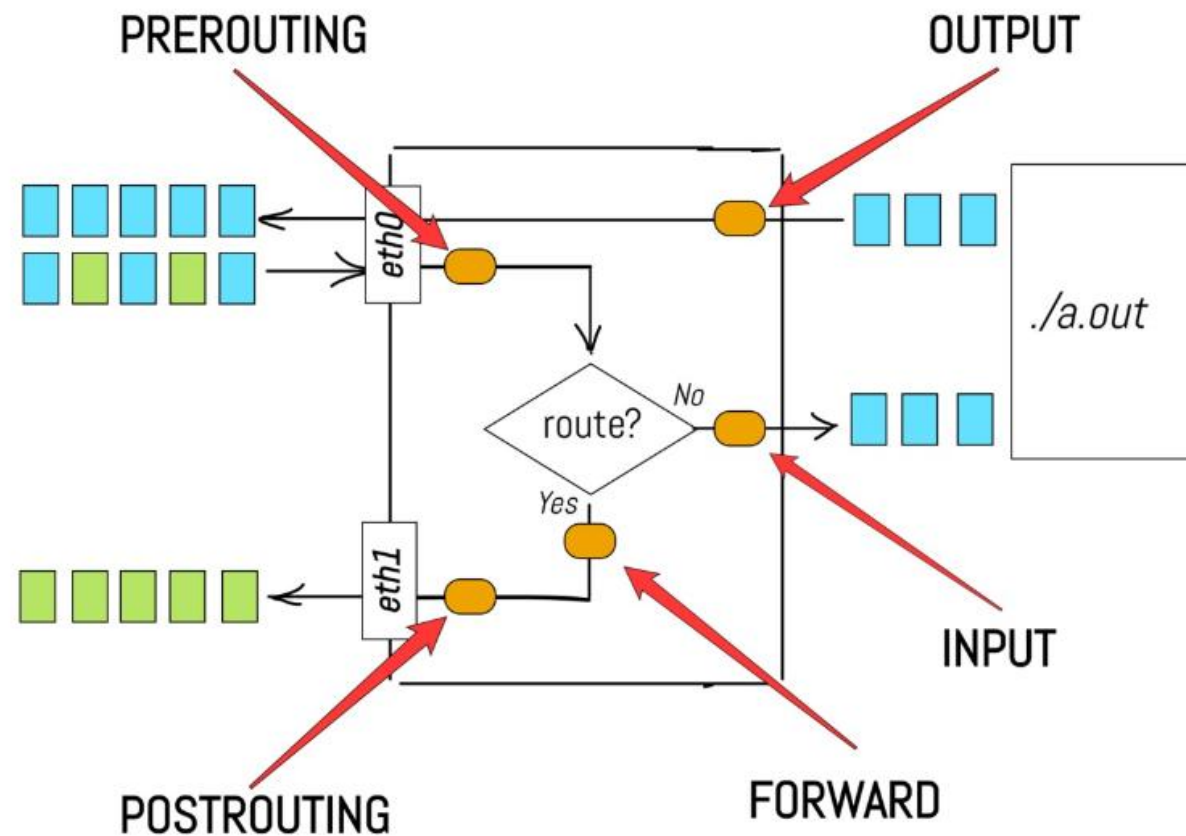
```
ip route show
```

```
ip route add 192.168.1.0/24 via 192.168.0.1
```



Linux routing

<https://iximiuz.com/en/posts/laymans-iptables-101/>



Virtual Interfaces and Bridges

Virtual Interfaces

- Loopback (lo) interface
- VLAN interfaces (vlanXX)
- TUN/TAP interfaces for virtual networking

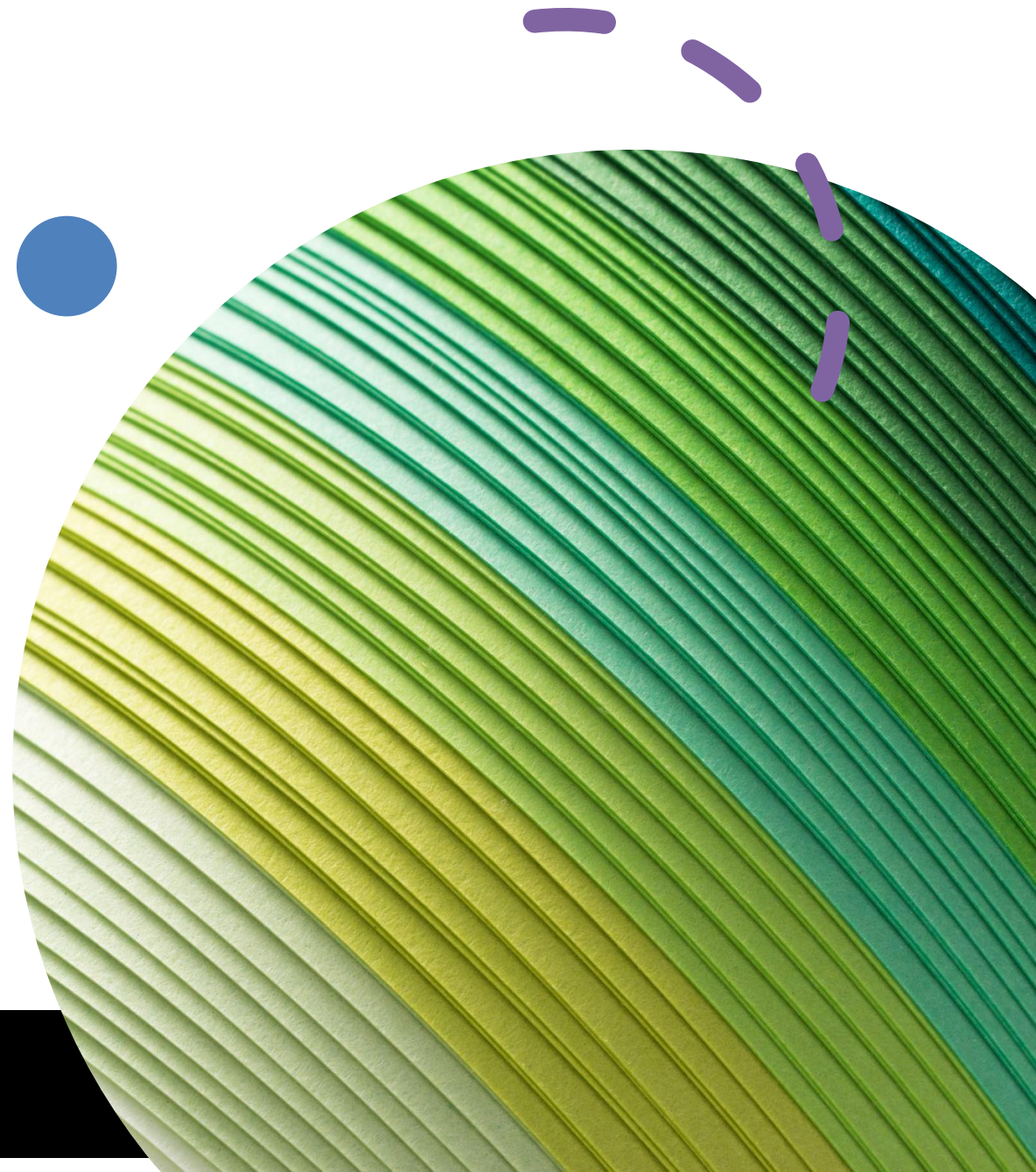
Bridges

- Layer 2 switching between interfaces
- Used in virtualization (e.g., brctl, ip link add br0 type bridge)
- Example:

```
brctl addbr br0
```

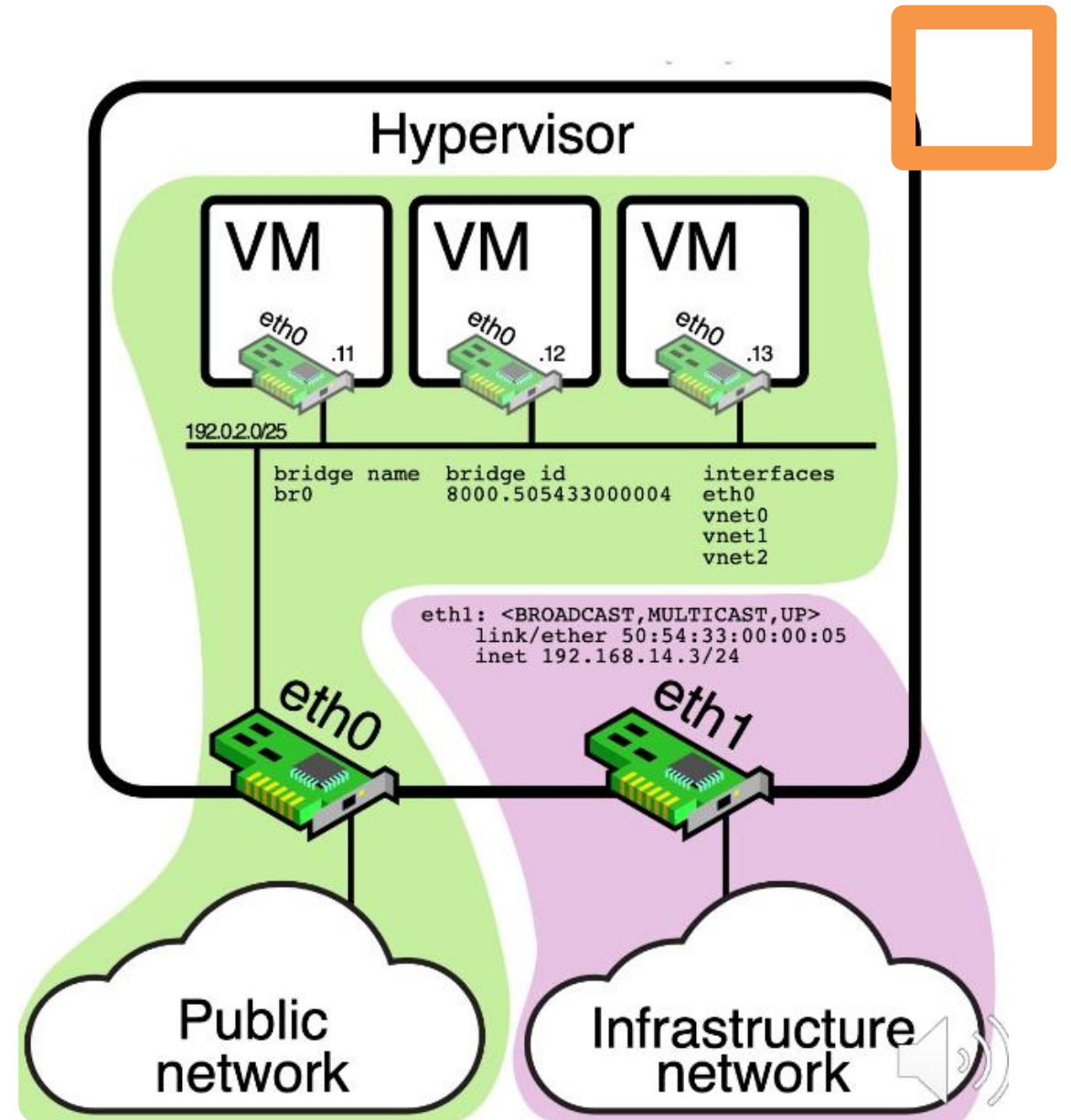
```
brctl add if br0 eth0 eth1
```

```
ip link set br0 up
```



Virtual Interfaces and Bridges

<https://vincent.bernat.ch/en/blog/2017-linux-bridge-isolation>



Network Namespaces

What are Network Namespaces?

Provides isolated network environments within the same OS

Used in containers and network sandboxing

Managing Namespaces

Creating a namespace:

```
root@host:/# ip netns add mynamespace
```

Running commands inside a namespace:

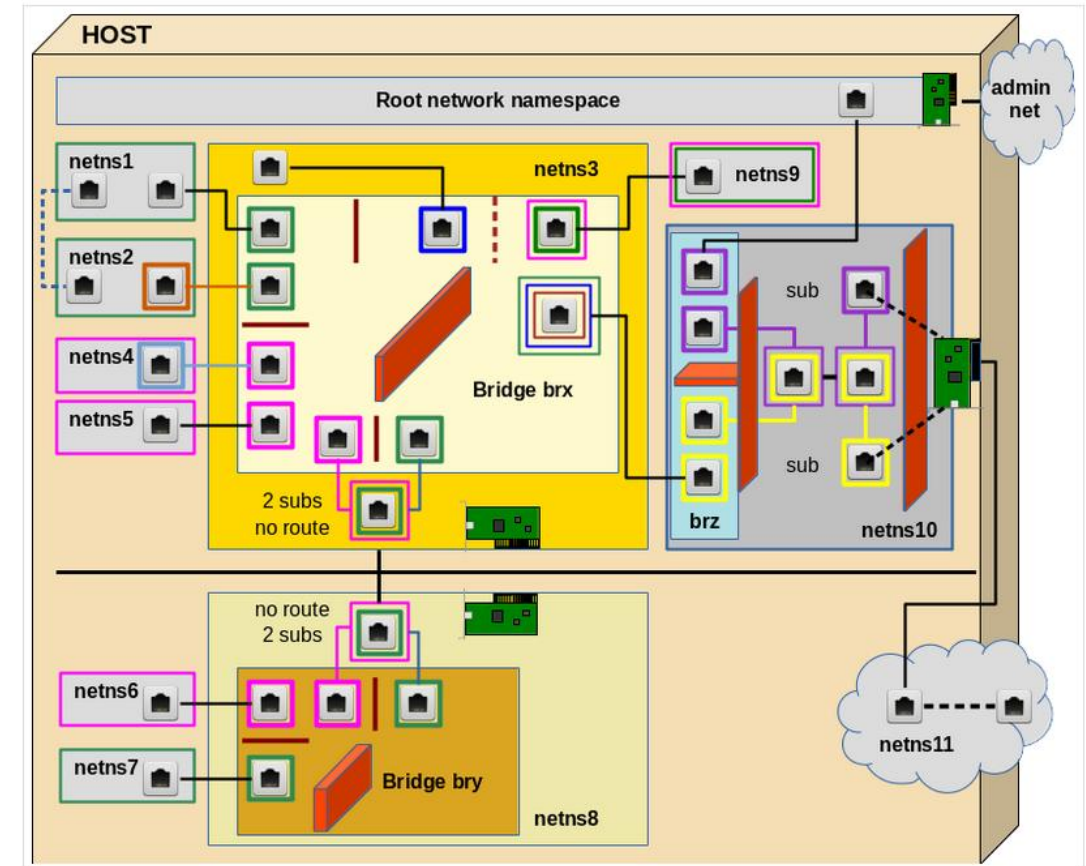
```
root@host:/# ip netns exec mynamespace ip addr
```

Deleting a namespace:

```
root@host:/# ip netns del mynamespace
```

Network Namespaces

<https://linux-blog.anracom.com/2017/11/12/fun-with-veth-devices-linux-bridges-and-vlans-in-unnamed-linux-network-namespaces-ii/>



Python for Network Automation



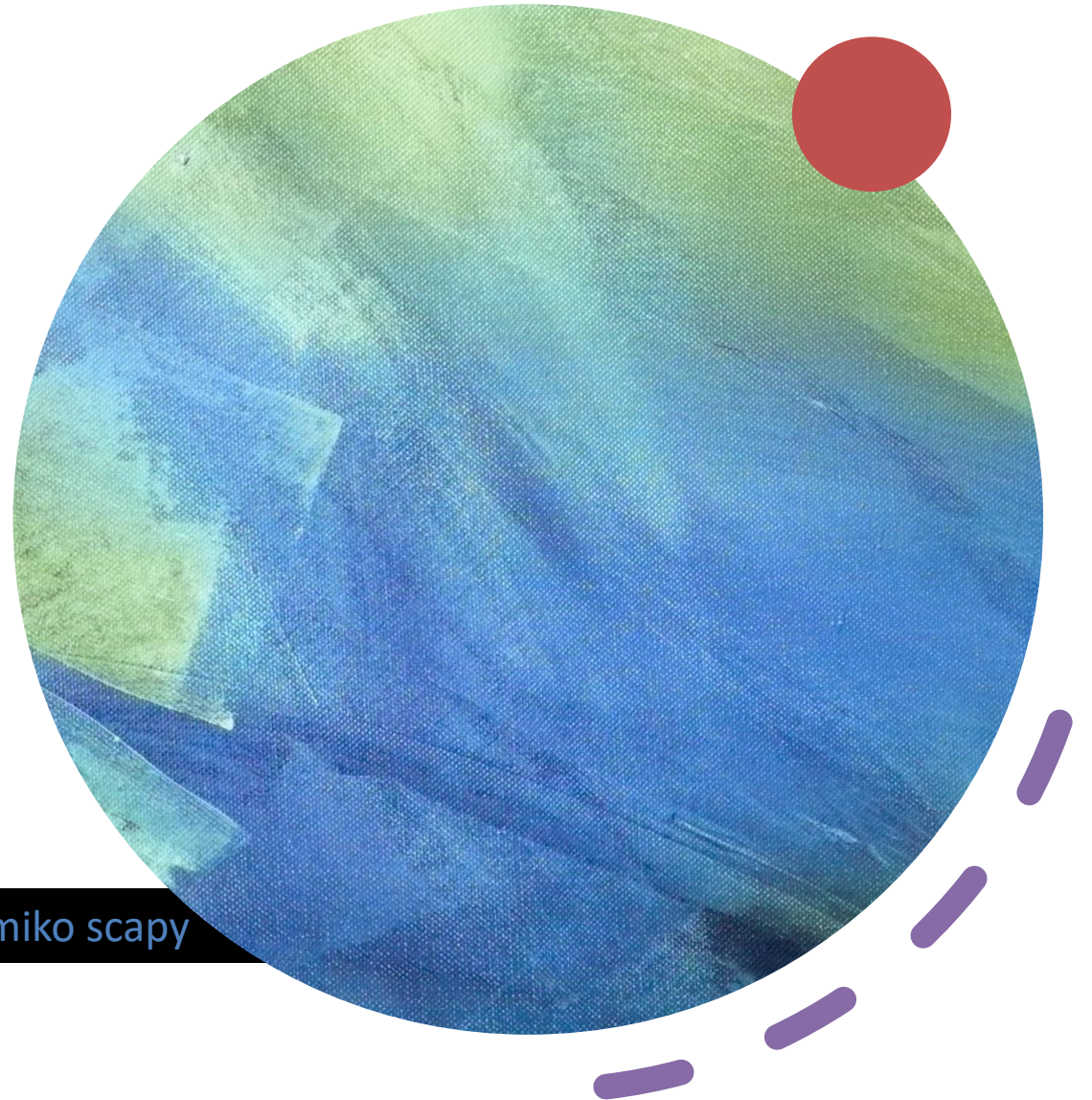
Why use Python for Networking?

- Automates repetitive networking tasks
- Interfaces with Linux networking tools
- Supports libraries like paramiko, netmiko, scapy



Installing Required Libraries

```
pip install paramiko netmiko scapy
```



Network Source of Truth

Reference of network assets and intended configurations

Used as input for configuration etc.

Manually ok for small deployments

```
router['interfaces'] = ['eth0', 'eth1']
router['IP'] = {'eth0': '10.1.1.254', 'eth1': '10.1.2.254'}
```

Or code your way around big deployments

```
hosts = []
for i in range(0,20):
    Host_i = {'hostname':'ws' + str(i) + '.xpto.pt',
              'IP':'10.1.2.' + str(i) + '/24',
              'if':'eth1', 'gw': '10.1.2.254', 'host_i':i}
    hosts.append(host_i)
```

Prepare Configurations

Define
function that
sets ip address
on interface

```
def set_ip_addr (if, ip):  
    cmds = [  
        'ip link set ' + if + ' up',  
        'ip address add ' + ip + ' dev ' + if  
    ]  
    return cmds
```

Host
interfaces

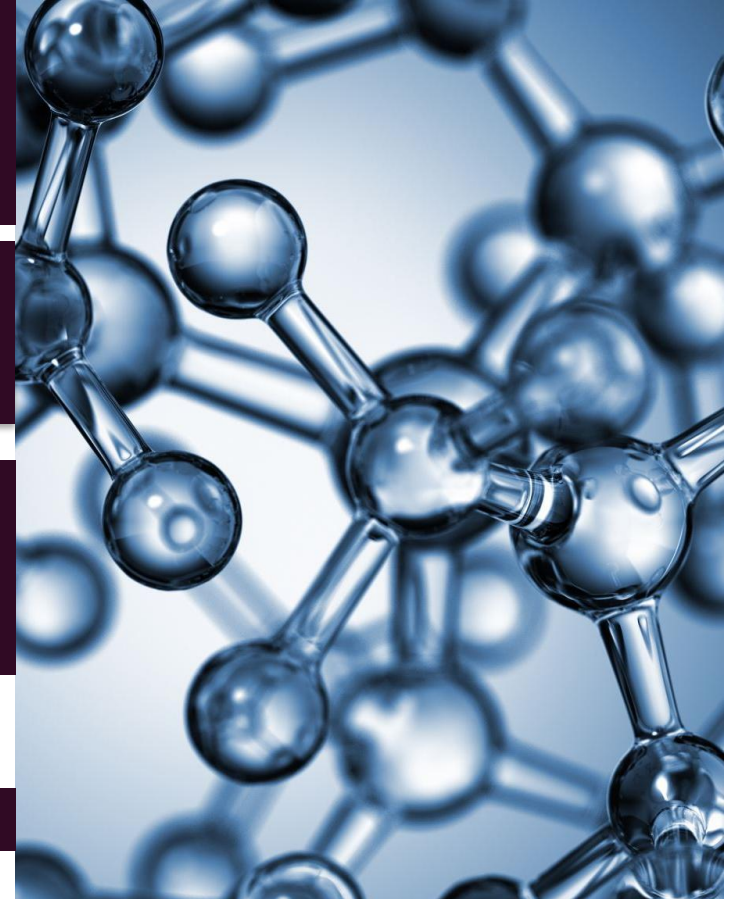
```
host_cmds = {}  
for host in hosts:  
    cmds = set_ip_addr (host['if'], host['IP'])  
    host_cmds[host['hostname']] = cmds
```

Router
interfaces

```
router_cmds = [  
    'sysctl -w net.ipv4.ip_forward=1',  
]  
for if in router['IP'].keys():  
    router_cmds.extend(set_ip_addr(if, router['IP'][if]))
```

Sanity check

```
print (host_cmds, router_cmds)
```



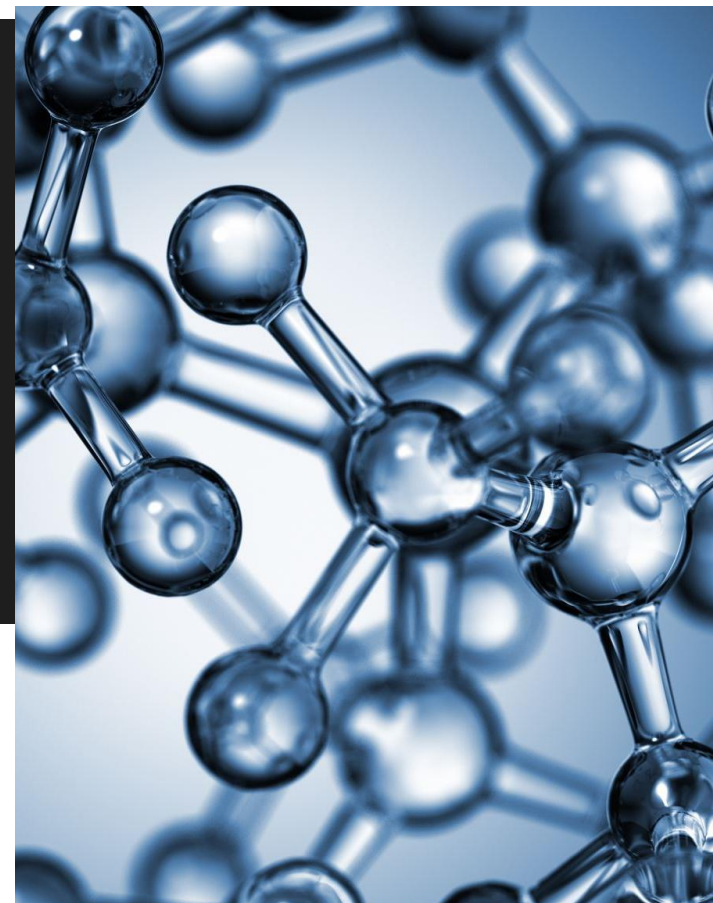
Network Device Configuration with Netmiko

Using Netmiko
for configuring
routers/switches

```
from netmiko import ConnectHandler

device = {
    'device_type': 'cisco_ios',
    'host': '192.168.1.1',
    'username': 'admin',
    'password': 'password',
}

connection = ConnectHandler(**device)
output = connection.send_command('show ip interface brief')
print(output)
connection.disconnect()
```



Automating Configuration with Jinja2

Example: Cisco Router Configuration Template

```
from jinja2 import Template

template = Template('''
interface {{ interface }}
    ip address {{ ip }} {{ mask }}
    no shutdown
''')

config = template.render(interface="GigabitEthernet0/1", ip="192.168.1.1", mask="255.255.255.0")
print(config)
```

Example: Mikrotik Router Configuration Template

```
template = Template('''
/ip address
add address={{ ip }}/{{ mask }} interface={{ interface }}
''')

config = template.render(interface="ether1", ip="192.168.2.1", mask="24")
print(config)
```

Packet Manipulation with Scapy

Using
Scapy to
craft
and
analyze
network
packets

```
from scapy.all import *  
  
packet = IP(dst="8.8.8.8")/ICMP()  
response = sr1(packet, timeout=2)  
  
if response:  
    print("Received response from", response.src)
```



Further Reading & References

- man ip, man brctl
- Linux kernel networking documentation
- Python networking libraries: paramiko, netmiko, scapy, jinja2
- <https://www.packetswitch.co.uk/generating-cisco-interface-configurations-with-jinja2/>

